A NON-CUSTODIAL RENT MANAGEMENT SYSTEM FOR INDEPENDENT LANDLORDS IN INDIA: DESIGN, IMPLEMENTATION AND VERIFICATION

Anush Kulal M
24MCAR0133
POST GRADUATE STUDENT (MCA)
JAIN (DEEMED-TO-BE UNIVERSITY), BENGALURU

Dr. Kamalraj R
Professor, Department of Computer Science and IT
JAIN (Deemed-to-be University), Bengaluru, India

ABSTRACT

Rented accommodation houses a large share of students and working people in Indian cities, and most of it is still run on paper. Rent arrives by UPI or in cash, the only record of a payment is a screenshot on the payer's phone, and neither side holds an account of the tenancy that the other has agreed to. The software built for this market divides into two kinds: digital registers with no tenant-facing side, which leave the tenant exactly where the paper register left them, and platforms that solve the record problem by collecting the rent themselves, which places the operator inside the Reserve Bank of India's payment-aggregator perimeter and routes a landlord's income through an account that is not theirs. This paper reports the design, implementation and verification of TenantPro, a two-sided mobile system that gives both parties one shared record while never receiving a rupee. Rent moves directly from tenant to landlord, by UPI or through a gateway on the landlord's own merchant account, and the platform states the amount and the destination and records what happened. Because a system that does not handle money cannot observe it, a payment reported by a tenant is treated as a claim that advances no rent cycle until a second party vouches for it. The implementation runs to 55,499 lines, 96 endpoints and 23 tables, and is verified by 750 assertions across 22 suites, of which 260 drive the compiled application inside a browser engine. Verification produced two results that generalise beyond this system: a test that shares its subject's assumption cannot detect the defect that assumption causes, and checking a project's own documentation against its source located four defects that every suite had passed, one of which had been formally recorded as fixed.

KEYWORDS

Rent Management; Property Management System; Non-Custodial Payments; Payment Aggregator Regulation; UPI; End-to-End Testing; Documentation Verification; React Native; MySQL.

INTRODUCTION

Paying guest establishments, private hostels and small apartment blocks of between two and twenty rooms are where a large proportion of urban renters in India live. Almost all of these properties belong to individuals rather than to firms, and almost all are run without software of any kind. A register at the front desk records names. Rent comes by UPI transfer or in cash. Deposits are remembered rather than written down. Maintenance complaints arrive by telephone and are either dealt with or forgotten, with nothing recorded either way.

At one property with four rooms this is adequate. At two properties with twelve it is not, and it fails unevenly. The landlord loses the ability to answer the central question of the business, which is who has not paid. The tenant loses more. Having paid, they hold a transaction screenshot that proves money moved and says nothing about which month it was for, and they hold no acknowledgement from the person who received it.

The products already serving this market split along a line that runs through the whole category. The simpler ones are digital registers with no tenant-facing side at all. The more capable ones do give the tenant a view, and they make that view authoritative by collecting the rent themselves and settling it onward. That is clean as software and consequential as regulation: an entity that receives payments from customers and remits them to merchants is a payment aggregator under the applicable framework, which carries an authorisation requirement, a minimum net worth in the tens of crores of rupees, and an obligation to hold collected funds in escrow. It also means a landlord's rent spends time in an account that is not theirs, which landlords interviewed during this project objected to without being asked.

This paper describes a system built on the view that these are not the only two options available.

REVIEW OF LITERATURE

Management literature on small enterprises: Work on why small businesses remain on manual systems finds consistently that the barrier is not the price of software but the time and risk of migration, and that manual systems fail quietly. A missed payment, a partial payment nobody followed up, or a deposit returned in full when deductions were due leave no trace, so the owner's sense of the business is reliably better than its accounts would be. The design conclusion is that any replacement must be usable without a migration project.

Existing digital rental products: A review of applications targeting the Indian rental market showed the split described above, and a second weakness in the simpler products: several model a property as a flat list of tenants with no intervening notion of rooms, which fails as soon as an owner holds more than one building.

Payment aggregator regulation: The Reserve Bank of India regulates entities that collect payments from customers and pass them to merchants. The framework has been revised more than once and the direction of travel has been to bring more arrangements inside it. The consequence for a small rental application is blunt: the moment the platform receives a

tenant's rent and later pays it to the landlord, it is doing the regulated thing, whatever it calls itself and however briefly it holds the money.

Data protection: The Digital Personal Data Protection Act, 2023 imposes purpose limitation and data minimisation, with obligations commencing in stages. For a system of this kind the practical effect is on two things: what a registration form may ask for, and how long an identity document may remain readable after the relationship that justified collecting it has ended.

Identity documents: The Aadhaar framework restricts publication of identity numbers and limits who may authenticate against the national database. Private-entity authentication was struck down in 2018 and later reopened on a narrower, permission-based route. A system that does not need to authenticate against a national database is therefore better advised not to.

Capacity to contract: Under the Indian Contract Act, 1872 a person who has not attained majority is not competent to contract, so a tenant under eighteen cannot hold a tenancy in their own name. This is not an academic point for student accommodation, where a proportion of residents are in their first year of higher education and have not yet turned eighteen.

Session security: Published guidance on token-based sessions recommends short-lived access tokens paired with rotating refresh tokens, with reuse of a spent refresh token treated as evidence of theft rather than as an ordinary error.

Research gap: The regulatory and security literature describes constraints well and the product literature describes features well, but neither addresses the question this project sets: whether a complete, shared and trustworthy tenancy record can be maintained by a platform that never receives the money, and what such a system must do differently as a result.

OBJECTIVES

To replace manual record keeping with a structured record of properties, rooms, tenants, tenancies, rent, deposits and payments, held in a relational database rather than in a register.

To give the tenant a real view of that record, so that the person paying the rent can see what is owed, what has been paid and what the landlord has formally acknowledged.

To keep the platform entirely outside the flow of funds, so that it never holds or transmits a tenant's money and does not fall inside the regulatory perimeter that applies to payment aggregators.

To ensure that no payment is treated as settled on the payer's own say-so, by requiring confirmation from the landlord or from a payment gateway before a rent cycle advances.

To verify the result by driving every layer against real infrastructure rather than against mocks, and to report what that verification found, including what it did not cover.

DATA COLLECTION

Source of evidence

The evidence in this paper is the system itself and the measurements taken from it. There was no questionnaire and there are no human respondents; the units of observation are source files, endpoints, database tables, test assertions, defects and elapsed times.

All counts were taken mechanically from the repository at a single commit, so that they describe one state of the system rather than an accumulation across several.

Method of verification

The build carries 750 assertions across 22 suites. None uses a test framework; each suite is a plain script that exits non-zero when an assertion fails, which removes any dependency between the suites and a runner's conventions.

End-to-end suites run against a live server and a real MySQL instance seeded by a generator with a fixed seed, so that a failing run is exactly reproducible.

The generator deliberately produces records that break interfaces and seldom appear when sought: a tenant with no room, a guest with no email address, rent overdue and due today and never configured, a payment declared and undecided, another rejected with a reason, and a property containing no rooms.

User-interface suites execute the same JavaScript the handset executes, exported for the web and driven through a browser engine, so that the assertions concern the compiled application rather than a reimplementation of it.

Measurement

Propagation latency was measured by a suite that opens two browser contexts, signs a tenant into one and a landlord into the other, and times the interval between a maintenance request being raised and its becoming visible to the other party.

Defects are counted as distinct causes, not as failing assertions, and each is counted once irrespective of how many suites observed it.

Endpoint coverage was computed by comparing the distinct paths requested by the suites against the routes the server declares.

Limits of the evidence

Nothing behind a native module is exercised, which excludes the camera, the image picker, push delivery and the handoff to a UPI application. These are stated rather than implied, because a passing run is persuasive.

Running a desktop browser is not a substitute for a device and is not claimed as one.

DATA ANALYSIS

The tables below describe the scale of the implementation, the distribution of verification effort across it, and what that verification found.

Table 1. Scale of the implementation

Particulars	Measure	Value
Source	Source files	178
Source	Lines of source	55,499
Interface	API endpoints declared	96
Interface	Screens	28
Data	Database tables	23
Verification	Test suites	22
Verification	Assertions	750

Source: Counted from the repository at the commit documented; author's computation.

Table 2. Distribution of assertions across verification layers

Layer	What it exercises	Suites	Assertions	Percent
Pure and stubbed	Functions in isolation	8	305	40.7
Live server	Endpoints against real MySQL	3	185	24.7
Browser-driven	The compiled application	11	260	34.7
Total	—	22	750	100.0

Source: Assertion counts as printed by a passing run of each suite; author's computation.

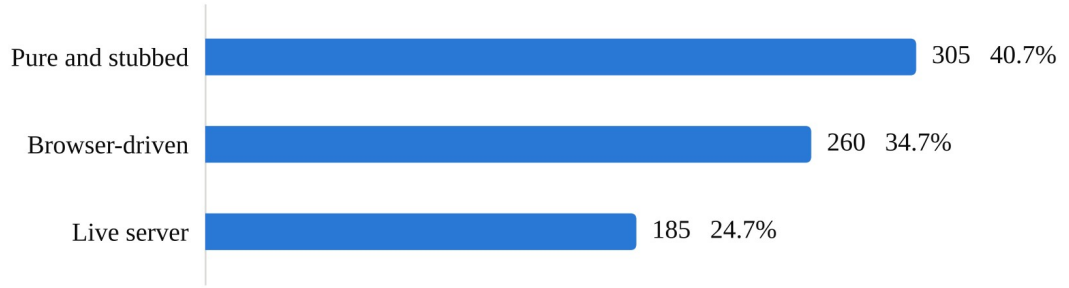


Figure 1. Assertions by verification layer (n = 750)

Source: Table 2. Eight suites run in isolation, three against live MySQL and eleven against the compiled application.

Table 3. Defects located, by class

Class	Count	Example

Correctness	6	A path promising a message the system could not send
Accessibility	3	Icon-only controls announcing only as "button"
Robustness	3	A missing required field answered with a server error
User-facing	2	A wrong password reported as an expired session
Authorisation	1	An endpoint omitting an ownership check
Security	1	One registration endpoint enforcing no password length
Total	16	—

Source: Defects recorded during verification; classified by the author.

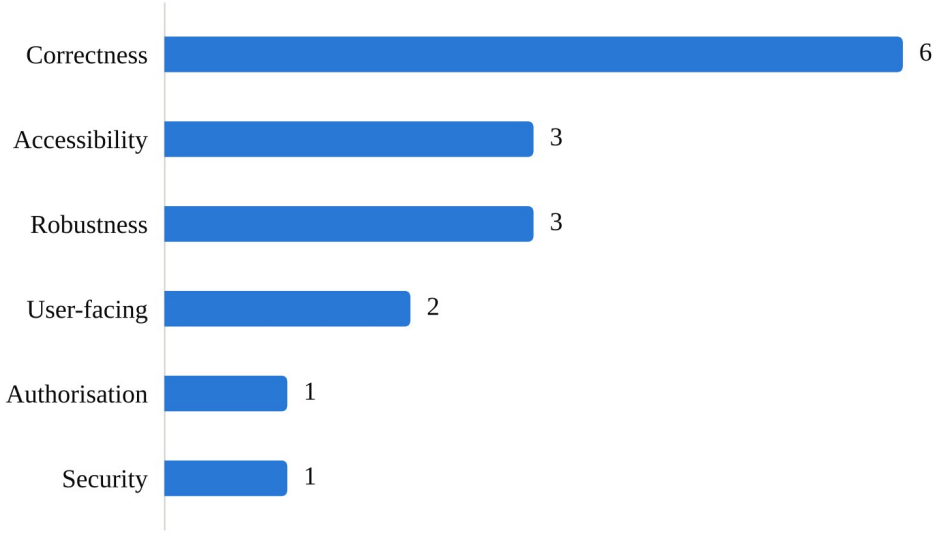


Figure 2. Defects located during verification, by class (n = 16)

Source: Table 3. None of the sixteen was detected by the eight suites that run in isolation.

Table 4. Cross-portal propagation latency

Condition	Observed latency	Polling interval	Within interval
Landlord interacting with the interface	2.3 s	25 s	Yes
Landlord's screen untouched	9.1 s	25 s	Yes

Source: Measured by the two-context suite; mean of observed runs. Author's measurement.

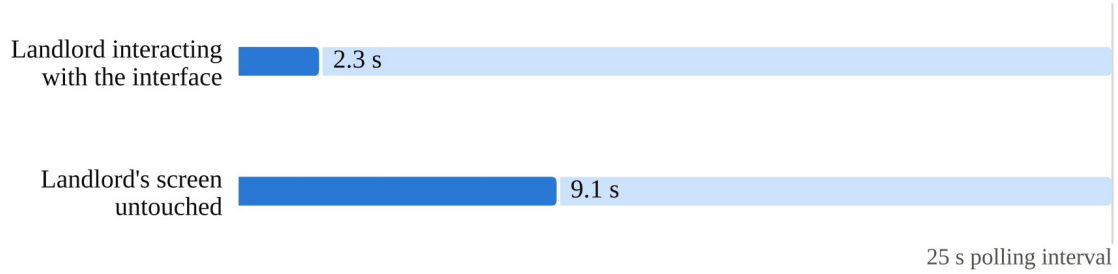


Figure 3. Propagation latency against the 25-second polling interval

Source: Table 4. Both conditions fall inside the interval; the first is faster because any navigation refetches.

Table 5. Endpoint coverage by the verification suites

Measure	Count	Percent of declared
Routes declared by the server	96	100.0
Distinct paths requested by the principal API suite	50	52.1
Routes not driven by any suite	approximately 32	33.3

Source: Computed by comparing requested paths against declared routes; author's computation.



Figure 4. Endpoint coverage against the 96 routes the server declares

Source: Table 5. About a third of declared routes are driven by no suite; the second figure is approximate at source.

Table 6. Test suites in the build

Category	Suite	Assertions
Pure and stubbed	secretbox, session, gateway, vm.gateway, refresh.client, layout, ratelimit, appversion	305
Live server	api.e2e, refresh.e2e, guardian.e2e	185
Browser-driven	ui.e2e, ui.crossportal,	260

	ui.discover, ui.alerts, ui.tour, ui.signup, ui.signuproles, ui.updates, ui.storebanner, ui.layout, ui.signout	

Source: Suite inventory taken from the repository; assertion counts as printed by a passing run.

Table 7. Defects located by checking documentation against source

Defect	Note
The guided tour did not start after registration	Recorded as fixed; three of four entry points had been addressed
Guardian details were stored but retrieved by no query	The interface's undertaking remained false
The update prompt had stopped listing an update's contents	Introduced by a refactor three days earlier
One suite loaded a dependency by an absolute path	Executable on one machine only

Source: Thirty statements in the project's documents were checked against the source; four failed because the software was wrong. Authors' verification.

Table 8. Objectives and the mechanism that meets each

Objective	Mechanism in the implementation
Structured record in place of a register	23 tables holding properties, rooms, tenancies, rent, deposits and payments
A real tenant-facing view	A tenant portal presenting the same records from the other side
Platform outside the flow of funds	UPI to the landlord's own address, or a gateway on the landlord's own merchant account
No payment settled on the payer's say-so	Claim-and-confirmation, with one shared settlement function
Verification against real infrastructure	185 assertions against live MySQL and 260 against the compiled application

Source: Mapping prepared by the author from the implementation.

PROPOSITIONS AND THEIR TESTS

This study has no survey and therefore no statistical hypotheses. What it has instead are propositions about the system that can be tested against the measurements above, and the three that carried the most weight for the design are set out here with the evidence that bore on them.

Proposition 1: A shared tenancy record can be maintained without the platform receiving funds.

Claim: A system that never receives money can nonetheless maintain a record both parties regard as authoritative.
Evidence: The implementation maintains such a record across 23 tables and 96 endpoints with no account belonging to the operator appearing in any payment path. Authority is supplied not by custody but by the confirmation model: a payment reported by a tenant advances no rent cycle until the landlord confirms it or a gateway answers a direct query, and both routes terminate in one settlement function so that they cannot disagree.
Result: Supported. The cost of the approach is explicit rather than hidden, which is that the system cannot observe a payment it did not handle, and the confirmation step exists to discharge that cost rather than to conceal it.

Proposition 2: Unit testing over mocks detects the defects that end-to-end execution detects.

Claim: If the units are correct, the assembled system is correct.
Evidence: Sixteen distinct defects were located by driving endpoints against a real database and the compiled application against a real browser engine. None had been detected by the unit suites, which passed throughout. The classes are given in Table 3 and include a cross-account authorisation defect in which one landlord's tenant could be written into another landlord's room.
Result: Not supported. The defects clustered precisely where units meet: authorisation applied in two of three places, a form and a server disagreeing about a rule, and an interface promising something no service implemented.

Proposition 3: A test suite detects defects arising from assumptions it shares with its subject.

Claim: Verification is independent of the thing verified.
Evidence: The client constrains keyboard-driven scrolling so that a focused input cannot pass beneath the header, computing the prohibited region as the safe-area inset plus the header's height. The suite responsible measured the header the same way, as the uppermost fifty-eight pixels of the window. When a notice bar was introduced above the header, both became wrong in the same direction for the same reason, and the resulting defect was invisible to the suite.
Result: Not supported. Detection required a third element that had not previously existed. A test sharing its subject's assumptions does not verify those assumptions, and a passing run does not disclose which assumptions it is failing to examine.

Table 9. Summary of propositions tested

Proposition	Evidence relied on	Result
A shared record can be maintained without receiving funds	23 tables, 96 endpoints, no operator account in any payment path	Supported

Unit testing over mocks detects what end-to-end execution detects	16 defects, none detected by the unit suites	Not supported
A suite detects defects arising from assumptions it shares	The focus-clamp defect, invisible to the suite that shared its premise	Not supported

Source: Authors' analysis of the measurements reported in Tables 1 to 5.

FINDINGS

The non-custodial model is workable: A complete two-sided record was maintained with no platform account in any payment path, which keeps the operator outside the payment-aggregator perimeter and leaves settlement where the landlords consulted wanted it.

Confirmation is structural, not incidental: Because the platform cannot see money it does not handle, the claim-and-confirmation step is not a convenience feature but the mechanism by which the record becomes trustworthy. Three write paths clear a rent cycle, and the one exempted from confirmation is the landlord's own entry, because it is their statement about their own money and no second party remains to verify it against.

Regulation produced design, not commentary: Each constraint reviewed produced a specific and observable decision. Data minimisation explains why the registration form was not widened. The Aadhaar framework explains why no identity number is a field and no national authentication is attempted. The Contract Act explains why registration asks about majority and records who consents for a minor.

End-to-end execution found what units did not: All sixteen defects in Table 3 were located outside the unit layer, and the unit suites passed throughout. The strongest single instance was an authorisation defect reachable only by composing two legitimate operations.

A promise can survive two attempts to fix it: The under-eighteen path first asked the question and did nothing with the answer, then collected a guardian's details and stored them where no query retrieved them, leaving the interface's undertaking false in both cases. Only the third revision, which surfaces the guardian where the landlord decides, made it true.

Documentation checked against source is verification: Checking the project's own documents against the code located four defects that every suite had passed, including one that had been recorded as fixed in both a report and a commit message when three of its four cases had been addressed.

Coverage must be stated, not implied: The suites request about fifty of ninety-six declared routes, and roughly a third of the surface is driven by nothing. The routes covered were chosen because they carry money, authorisation or session state, which is the right priority, but a claim that all endpoints are tested would be false.

Suggestions

For independent landlords: a shared record is obtainable without surrendering custody of rent, and the confirmation step that such a system requires is a small cost against an auditable account of every month.

For builders of two-sided platforms in regulated areas: establish early whether the design places the operator inside a regulatory perimeter, because that determination constrains architecture rather than configuration.

For project verification: drive the assembled system against real infrastructure, state explicitly what the suites cannot reach, and check documentation against source as a distinct exercise.

Limitations

The system cannot observe money, which follows from the design rather than from an oversight. Session revocation lags by up to two hours on the access path. Push delivery to a handset has never been confirmed: the path is implemented and no device has registered against it, so the in-application notification surface is verified and the device notification tray is not. No iOS build has been produced. A second payment provider is specified and not implemented. Finally, the demonstration listings that populate the discovery interface on a new deployment carry ordinary names and stock photographs and are not marked as samples anywhere a user can see, which requires a visible marker.

CONCLUSION

This study shows that an independent landlord need not choose between a paper register and handing custody of their rent to a platform. A two-sided system can give both parties a complete, shared and trustworthy record of a tenancy while leaving the money exactly where it already flows, provided it accepts the consequence of that choice and builds the confirmation model that the consequence requires.

The verification work produced results of wider application than the system itself. Tests that assert the absence of a string, systems examined only against their own mocks, and tests that share their subject's assumptions all produce the same false confidence, and a passing run discloses which of the three is occurring in none of the cases. Checking documentation against source proved inexpensive and located four defects that every suite had passed, including one in a module written, tested and released three days earlier. A defect recorded as fixed is more damaging than one recorded as open, because it ends the inquiry that would otherwise have found it.

REFERENCE

Expo. (2025). Expo Updates and EAS Build documentation. Expo Documentation.

Government of India. (1872). The Indian Contract Act, 1872, s. 11.

Government of India. (2016). The Aadhaar (Targeted Delivery of Financial and Other Subsidies, Benefits and Services) Act, 2016, s. 29.
Government of India. (2023). The Digital Personal Data Protection Act, 2023, and the rules issued thereunder.
Internet Engineering Task Force. (2015). RFC 7519: JSON Web Token (JWT).
Internet Engineering Task Force. (2024). OAuth 2.0 Security Best Current Practice.
National Institute of Standards and Technology. (2007). SP 800-38D: Recommendation for block cipher modes of operation — Galois/Counter Mode (GCM).
National Payments Corporation of India. (2024). UPI linking specification and deep linking guidelines.
Nielsen, J. (1993). Usability engineering. Morgan Kaufmann.
Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.
Razorpay. (2025). Payment Links API. Razorpay Developer Documentation.
Reserve Bank of India. (2020). Guidelines on regulation of payment aggregators and payment gateways, and subsequent amendments.
World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2.